

Lab4- Flow Matching

111550132 張家睿

1. Introduction

- Brief overview of Flow Matching

Flow matching is a generative modeling framework that learns a time-dependent velocity field $v_0(x,t)$ which transports a simple source distribution to a complex target data distribution by solving an ordinary differential equation. Instead of learning a score function as in DDIM, flow matching directly parameterizes the dynamics of probability mass under a continuity equation and matches these dynamics to a chosen probability path between the source and target distributions.

Rectified Flow further refines the learned flow so that sample trajectories become straighter in state space. Straighter trajectories allow accurate synthesis with fewer ordinary differential equation steps, reducing inference time while maintaining generation quality.

- Assignment objectives

Task 1: Implemented a 2D flow matching model and visualized learned trajectories.

Task 2: Extended flow matching to images with CFG and evaluated FID and sampling behavior.

Task 3: Implemented rectified flow via reflow data generation and compared FID, speed, and trajectory straightness.

2. Methodology

- Implementation details for each task

- Task 1:

Implement a conditional Flow Matching model. At each training step, we sample a data point x_1 , a prior sample $x_0 \sim N(0, I)$, and a time $t \sim U(0, 1)$, then construct an intermediate point on the path $x_t = (1-t)x_0 + tx_1$ using the scheduler's ψ_t . The ground-truth velocity is the constant vector $v = x_1 - x_0$, and the network $v_\theta(x_t, t, y)$ is trained using an MSE loss, which corresponds to the Conditional Flow Matching objective. For sampling, it is initialised from Gaussian noise and numerically integrates the learned ODE from $t=0$ to $t=1$ using a first-order Euler solver, updating $x_{t+\Delta t} = x_t + \Delta t \cdot v_\theta(x_t, t, y)$ over a fixed number of inference steps. Also support CFG by combining

unconditional and conditional velocity predictions.

■ Task 2:

Implement the Flow Matching (FM) model used to generate high-quality samples through an ODE-based sampling process. The key component is the FMScheduler, which defines the rectified flow interpolation path and provides the Euler ODE solver. At each training step, randomly sample a timestep $t \sim U(0,1)$, generate the intermediate point $x_t = (1-t)x_0 + t \cdot x_1$, and train the neural network to predict the constant target velocity $u_t = x_1 - x_0$. The loss is computed using MSE between the predicted and target velocities, following the Conditional Flow Matching objective.

During inference, we start from pure Gaussian noise and solve the ODE from $t=0$ to $t=1$ using first-order Euler integration. At each timestep, the network predicts the instantaneous velocity, and we update the sample using $x_{t+\Delta t} = x_t + \Delta t \cdot v_t$. We also implement CFG by combining conditional and unconditional velocity predictions.

■ Task 3:

Implement a data generator that simulates the learned Flow Matching ODE to create training pairs for rectified flow. Given a pretrained FM checkpoint, I first load the model onto the specified GPU and, if CFG is enabled, infer the number of semantic classes from the class embedding table. For each batch, define the latent shape and, when CFG is used, sample integer class labels uniformly from 1 to num_classes, leaving 0 as the null token. Instead of manually integrating the ODE, it calls fm.sample to obtain the full trajectory. From this trajectory, extract the initial state $x_0 = \text{traj}[0]$ (noise at $t=0$) and the final state $z_1 = \text{traj}[-1]$ (generated sample at $t=1$). For every element in the batch, save x_0 , z_1 , and the corresponding label as .pt tensors using a zero-padded index, and optionally convert z_1 to a PIL image for quick qualitative inspection. Finally, store a config.json and metadata.json in the output directory.

● Key equations and algorithms

■ Linear probability path:

$$x_t = (1 - t) x_0 + t \cdot x_1$$

■ Conditional Flow Matching loss:

$$L_{CFM} = v_{uncond} + s (v_{cond} - v_{uncond})$$

■ Euler ODE solver for sampling:

$$x_{t+\Delta t} = x_t + \Delta t \cdot v_t$$

■ Straight-line path for rectified flow:

$$x_t = (1 - t) x_0 + t \cdot z_1$$

- Architecture choices

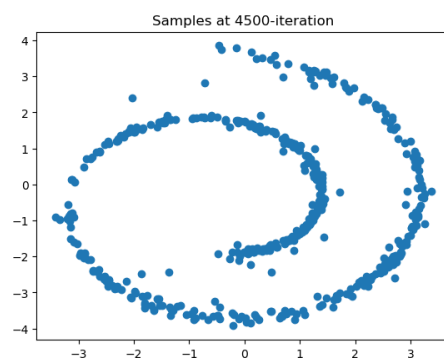
- Flow Matching:

The velocity network is the provided CNN-based backbone that maps an image x_t and a scalar timestep t to a velocity field of the same spatial shape. Time is encoded via a learnable embedding that is injected into intermediate feature maps, and class labels are embedded and added in the same way, so the network can learn time- and class-dependent flows without changing its convolutional structure. The core architecture follows an encoder-decoder design with convolutional blocks and skip connections, which is sufficient to capture local image structure while remaining lightweight.

3. Experiments & Results

- Task 1:

- Trajectory visualizations

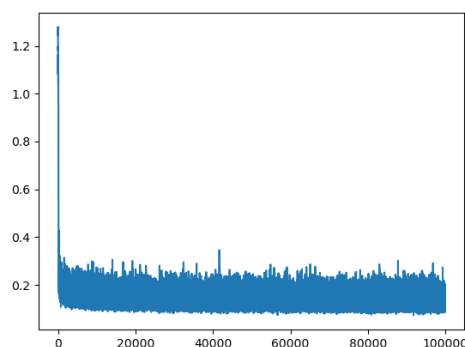


- Chamfer Distance

Chamfer Distance: 37.5498

- Task 2:

- Loss curves



■ FID vs CFG scale (num_inference_steps=20)

CFG	1.0	3.0	7.5
FID	13.56581	9.20830	9.64108

■ inference step analysis (cfg_scale=7.5)

inference step	10	20	50
FID	9.44234	9.64108	14.04284

■ Sample visualizations



● Task 3:

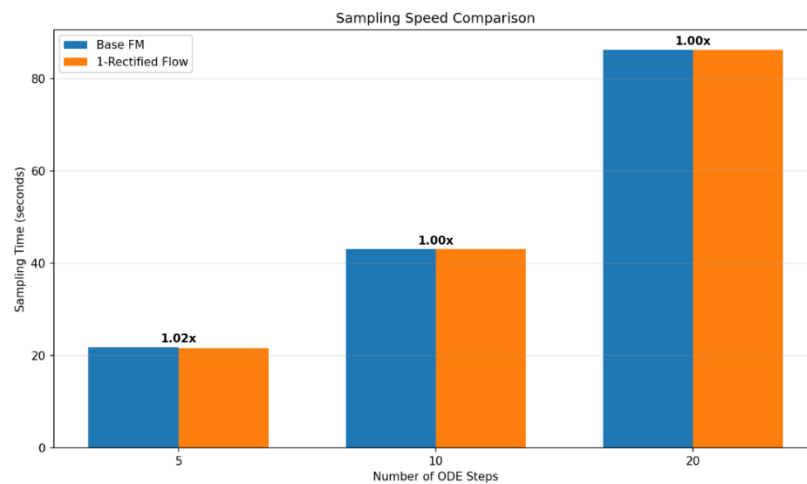
■ FID comparison at different inference steps

inference step	5	10	20
FID	8.60850	18.85662	20.95593

■ Comparison with Task 2 baseline (num_inference_steps=20)

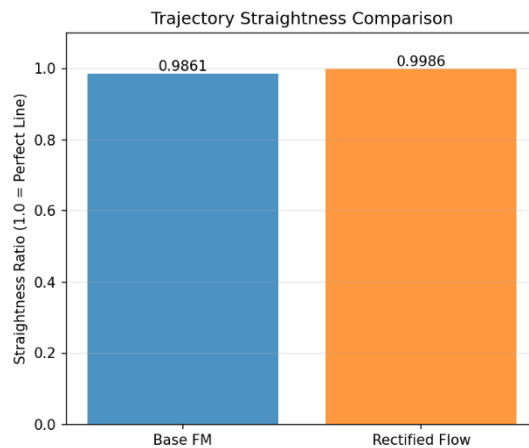
	Task2	Task3
FID	9.64108	20.95593

■ speedup analysis (wall-clock time measurements)



No significant improvement.

■ trajectory straightness



Trajectory straightness is computed as the ratio between the straight-line distance from the starting point to the ending point and the total path length traveled.

Concretely, each trajectory step is flattened into a vector, the Euclidean distance from the first to the last step is computed, then divided by the sum of Euclidean distances between every pair of consecutive steps, and finally this ratio is averaged over the batch (values closer to 1 mean a straighter trajectory).

4. Discussion

● Analysis of results

In terms of image quality, Flow Matching and Lab2's DDIM can achieve similar results, with feline images showing higher quality and canine images appearing less realistic.

Feline



Canine



- **Challenges encountered and solutions**
Using the hyperparameters in readme.md to train tasks 2 and 3 would take a lot of time. My approach is to copy the checkpoint halfway through training, rename it, and continue training while simultaneously testing the FID of that checkpoint. If the FID is high enough, further training is unnecessary. This saves time and addresses the issue of insufficient training time.
- **Answer discussion questions from each task**
 - **Discussion: Why does rectified flow enable faster sampling?**
Rectified flow enables faster sampling because it learns almost straight, well-conditioned trajectories from noise to data, so the ODE is much simpler to integrate. A simpler, less curved path means we can use far fewer time steps while still closely following the true trajectory, which keeps image quality high but makes generation much faster than standard diffusion.

5. Conclusion

- **Summary of findings**
Flow Matching can achieve similar generation results to DDIM, but it can use Rectified Flow to accelerate inference, so overall it can generate images of varying quality at a faster speed.
In the beginning, I think a larger inference step can lead to a better FID score, but it turns out it's not always the case. In both task 2 and task 3, lower inference steps resulted in better FID.
- **Future work suggestions**
Complete Task 4 to achieve true one-step generation.
Researching how to optimize the size of the Generate Reflow Dataset. The hyperparameters in the original readme.md generate 295GB of data, which is a bit too large.